

Foundations of Artificial Intelligence

M. Helmert
C. Büchner

University of Basel
Spring Term 2026

Exercise Sheet 11 — Solutions

Please note the submission rules at the end of the exercise sheet.

Exercise 11.1 (2+1 marks)

Consider the game *Nim* (<https://en.wikipedia.org/wiki/Nim>). The game starts with $n > 0$ stacks of coins where $c_i \geq 0$ denotes the number of coins on stack $1 \leq i \leq n$. Two players MAX and MIN alternate choosing a nonempty stack i and removing between 1 and c_i coins from it. Whichever player removes the last coin (leaving all stacks empty) wins the game.

- Formalize the game *Nim* with two stacks $c_1 = 2$ and $c_2 = 1$ as a zero-sum game wher MAX acts first.
- Draw the game tree for the game defined in part (a). Annotate terminal nodes with their utility.

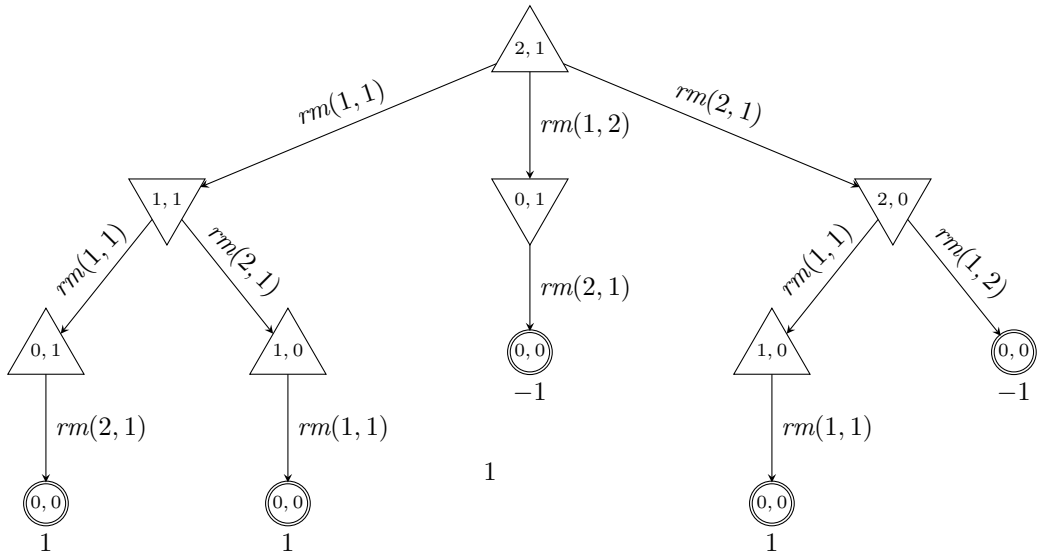
Solution:

- We define $\mathcal{S} = \langle S, A, T, s_I, S_G, utility, player \rangle$ where

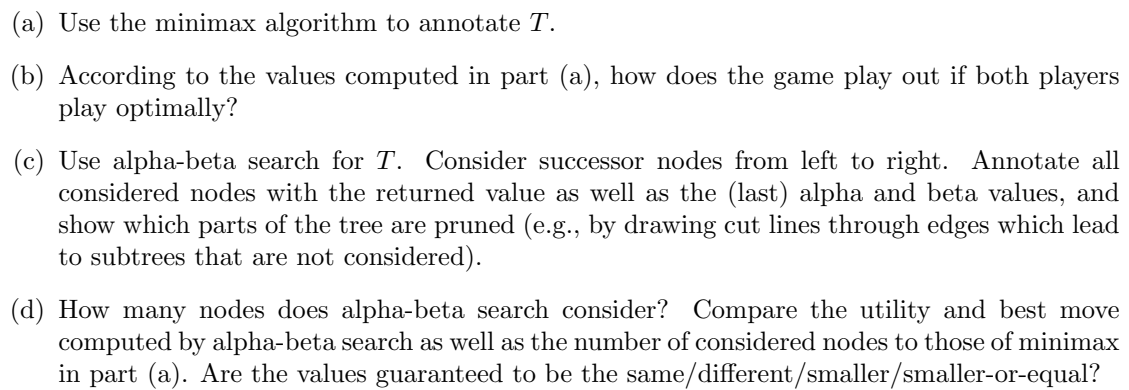
- $S = \{0, 1, 2\} \times \{0, 1\} \times \{\text{MAX}, \text{MIN}\}$,
- $A = \{rm(i, j) \mid i \in \{1, 2\}, 1 \leq j \leq c_i\}$,
- $T = \{ \langle \langle m_1, m_2, p \rangle, rm(i, j), \langle n_1, n_2, p' \rangle \rangle \mid 0 \leq m_1 \leq c_1, 0 \leq m_2 \leq c_2, m_1 + m_2 > 0, n_i = m_i - j, n_{3-i} = m_{3-i}, p' \in \{\text{MAX}, \text{MIN}\} \setminus \{p\} \}$,
- $s_I = \langle 2, 1, \text{MAX} \rangle$,
- $S_G = \{ \langle 0, 0, p \rangle \mid p \in \{\text{MAX}, \text{MIN}\} \}$,
- $utility(\langle 0, 0, \text{MIN} \rangle) = 1$ and $utility(\langle 0, 0, \text{MAX} \rangle) = -1$, and
- $player(\langle x, y, p \rangle) = p$ for all $0 \leq x \leq c_1$ and $0 \leq y \leq c_2$ with $x + y > 0$.

Note that we represent the player whose turn it is within the state description. This is a bit unusual but we need to do so in order to define the utility function which depends on which player took the last turn.

- Here, we represent the game states with only the two numbers denoting the numbers of coins in each stack. This is to reduce representation size and because the player information is implicitly given by the node type (MAX in triangles pointing upward, MIN in triangles pointing downward).



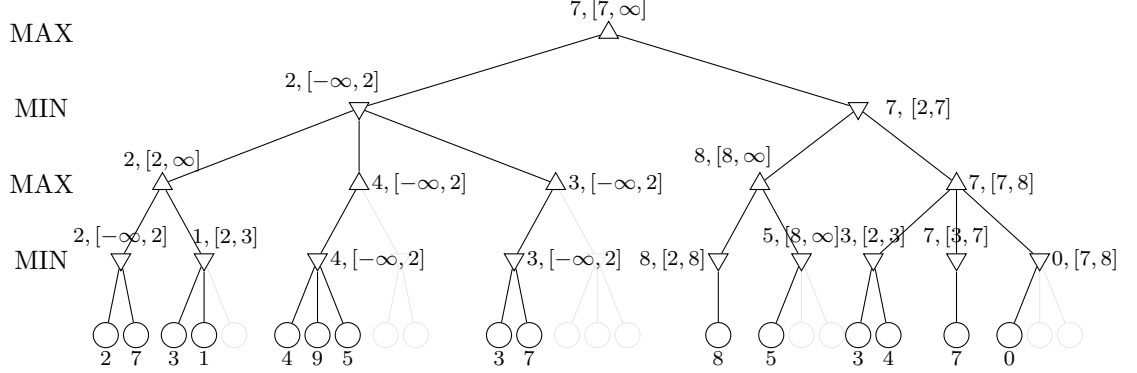
Consider the following game tree T :



(a) The annotated game tree looks like this:



(c) The annotated game tree looks like this:



(d) Alpha-beta search considers 32 nodes while plain minimax considers 44 nodes. The number of nodes that alpha-beta search has to consider is guaranteed to be less than or equal to the number of nodes considered by minimax. There is no guarantee that it is strictly less as in this case, though in practice this is usually the case. The returned utility value is 7 in both cases. Since alpha-beta search only prunes parts of the game tree that are irrelevant, the returned utility is guaranteed to be the same as for minimax. The returned best moves are identical if applicable moves are considered in the same order in both algorithms.

Exercise 11.3 (1 mark)

Consider the stochastic game $\mathcal{S} = \langle S, A, T, s_I, S_G, \text{utility}, \text{player} \rangle$ with

- $S = \{0, \dots, 44\} \times \{0, \dots, 27\} \times \{\top, \perp\} \times \{0, \dots, 27\} \times \{\top, \perp\}$
- $A = \{\text{roll}, \text{stop}\}$
- $T(\langle s, \text{roll}, t_r \rangle) = 1/6$ for $r \in \{1, \dots, 6\}$ for
 - positions $s = \langle k, v_x, a_x, v_y, a_y \rangle$ with $\text{player}(s) = \text{MAX}$ and moves $t_r = \langle k+1, v_x+r, a'_x, v_y, a_y \rangle$ where $a'_x = \perp$ iff $v_x+r > 21$.
 - positions $s = \langle k, v_x, a_x, v_y, a_y \rangle$ with $\text{player}(s) = \text{MIN}$ and moves $t_r = \langle k+1, v_x, a_x, v_y+r, a'_y \rangle$ with $a'_y = \perp$ iff $v_y+r > 21$.

$T(\langle s, \text{stop}, t \rangle) = 1$ for

- positions $s = \langle k, v_x, a_x, v_y, a_y \rangle$ with $\text{player}(s) = \text{MAX}$ and moves $t = \langle k+1, v_x, \perp, v_y, a_y \rangle$.
- positions $s = \langle k, v_x, a_x, v_y, a_y \rangle$ with $\text{player}(s) = \text{MIN}$ and moves $t = \langle k+1, v_x, a_x, v_y, \perp \rangle$.

$T(\langle s, a, t \rangle) = 0$ for all other combinations of $s, t \in S$ and $a \in A$.

- $s_I = \langle 0, 0, \top, 0, \top \rangle$
- $S_G = \{ \langle k, v_x, a_x, v_y, a_y \rangle \in S \mid a_x = a_y = \perp \}$
- $\text{utility}(\langle k, v_x, a_x, v_y, a_y \rangle) = \begin{cases} 1 & \text{if } v_y < v_x \leq 21 \text{ or } v_x \leq 21 < v_y \\ -1 & \text{if } v_x < v_y \leq 21 \text{ or } v_y \leq 21 < v_x \\ 0 & \text{otherwise} \end{cases}$

$$\bullet \text{ player}(\langle k, v_x, a_x, v_y, a_y \rangle) = \begin{cases} \text{MAX} & \text{if } a_x = \top \text{ and } (k \text{ is even or } a_y = \perp) \\ \text{MIN} & \text{if } a_y = \top \text{ and } (k \text{ is odd or } a_x = \perp) \end{cases}$$

Describe the rules of this dice game in natural language. What are the players trying to do?

Solution:

Two players take turns rolling a 6-sided die and sum up their results. They are trying to reach a result as close as possible to 21 without going over. Each player may stop at any time, in which case the other player just keeps rolling until they also stop. Players who exceeded a score of 21 are disqualified. Once both players have stopped or are disqualified, the game ends. If only one player is disqualified, the other player wins. If neither of the players is disqualified, the player closer to 21 wins. If both are disqualified or they have the same score, the game ends in a draw.

Exercise 11.4 (2 marks)

Provide pseudocode for the expectiminimax algorithm (Chapter G5) along the lines of pseudocode for minimax provided in Chapter G2.

Hint: Recall that $\text{SUCC}(p)$ in a non-stochastic game $\mathcal{S} = \langle S, A, T, s_I, S_G, \text{utility}, \text{player} \rangle$ denotes the set of moves applicable in position $p \in S$ and the corresponding successor positions according to T , i.e., $\text{SUCC}(p) = \{ \langle \text{move}, p' \rangle \mid \langle p, \text{move}, p' \rangle \in T \}$. If $\mathcal{S}$ is a stochastic game instead, T is a *transition function* that is well-defined for all $\langle p, \text{move} \rangle$. For a given pair of positions $p, p' \in S$ and a *move* $\in A$, $T(p, \text{move}, p')$ denotes the probability of obtaining p' from applying *move* in p : if *move* is not applicable in p , then $T(p, \text{move}, p') = 0$ for all p' , otherwise T with fixed p and *move* is a probability distribution over S .

For this exercise, assume there are functions $\text{APPLICABLE}: S \rightarrow 2^A$ and $\text{PROB}: S \times A \rightarrow S \times \mathbb{R}$ mapping states to sets of applicable actions and state-action pairs to the probability distribution of possible outcomes, respectively. They are formally defined as follows:

$$\begin{aligned} \text{APPLICABLE}(p) &= \{ \text{move} \in A \mid \exists p' \in S : T(p, \text{move}, p') > 0 \} \\ \text{PROB}(p, \text{move}) &= \{ \langle p', T(p, \text{move}, p') \rangle \mid p' \in S, T(p, \text{move}, p') > 0 \} \end{aligned}$$

Solution:

Algorithm 1 Pseudocode for expectiminimax given the following stochastic game: $\mathcal{S} = \langle S, A, T, s_I, s_G, \text{utility}, \text{player} \rangle$.

```

1: function EXPECTIMINIMAX( $p$ )
2:   if  $p$  is terminal position then
3:     return  $\langle \text{utility}(p), \text{none} \rangle$ 
4:    $\text{best\_move} := \text{none}$ 
5:   if  $\text{player}(p) = \text{MAX}$  then
6:      $v := -\infty$ 
7:   else
8:      $v := \infty$ 
9:   for  $\text{move} \in \text{APPLICABLE}(p)$  do
10:     $\text{expected\_}v' := 0$ 
11:    for  $\langle p', \text{prob} \rangle \in \text{PROB}(p, \text{move})$  do
12:       $\langle v', \text{best\_move}' \rangle := \text{EXPECTIMINIMAX}(p')$ 
13:       $\text{expected\_}v' := \text{expected\_}v' + \text{prob} \cdot v'$ 
14:    if ( $\text{player}(p) = \text{MAX}$  and  $\text{expected\_}v' > v$ ) or
       ( $\text{player}(p) = \text{MIN}$  and  $\text{expected\_}v' < v$ ) then
15:       $v := \text{expected\_}v'$ 
16:       $\text{best\_move} := \text{move}$ 
17:   return  $\langle v, \text{best\_move} \rangle$ 

```
